EXPERIMENT 3

ROLL NO: 240701020

NAME: ADITYA S

USER INTERFACE DESIGN

TITLE: Implementation of Windows Application Opener using CLI, GUI, and VUI in Python

1. COMMAND LINE INTERFACE (CLI)

Description

The Command Line Interface (CLI) allows the user to open Windows system applications by typing simple text-based commands in the terminal.
The program interprets the user command and launches the corresponding application using operating system calls.

This interface provides a fast and efficient way to perform Windows actions without using any graphical components.

Functionality

- Accepts command-based input from the user
- Opens built-in Windows applications
- Executes actions directly through the operating system

PYTHON CODE:

```
import os


def open_application(command):

    command = command.lower().strip()


    if command == "open notepad":
```

```
        os.startfile("notepad")
    elif command == "open calculator":
        os.startfile("calc")
    elif command == "open paint":
        os.startfile("mspaint")
    elif command == "open command prompt":
        os.startfile("cmd")
    elif command == "open control panel":
        os.startfile("control")
    elif command == "exit":
        return False
    else:
        print("Unknown command. Try: open notepad / open calculator")
    return True

def main():
    print("=== Windows Application Launcher (CLI) ===")
    print("Type commands like:")
    print("- open notepad")
    print("- open calculator")
    print("- open paint")
    print("- open command prompt")
    print("- open control panel")
    print("- exit")
```

```
    running = True
    while running:
        command = input("\nEnter command: ")
        running = open_application(command)


if __name__ == "__main__":
    main()
```

```
PS C:\Users\hp\Documents\User Interface Design\Design\UID EXPT 3> python cli.py
=== Windows Application Launcher (CLI) ===
Type commands like:
- open notepad
- open calculator
- open paint
- open command prompt
- open control panel
- exit

Enter command: open notepad

Enter command: open control panel

Enter command: leng
Unknown command. Try: open notepad / open calculator
```

```
PS C:\Users\hp\Documents\User Interface Design\Design\UID EXPT 3> python cli.py
=== Windows Application Launcher (CLI) ===
Type commands like:
- open notepad
- open calculator
- open paint
- open command prompt
- open control panel
- exit

Enter command: open paint

Enter command: exit
PS C:\Users\hp\Documents\User Interface Design\Design\UID EXPT 3>
```

Example Commands

- `open notepad`
- `open calculator`
- `open paint`

- `open command prompt`

OUTPUT

- The requested Windows application opens immediately
- Error message is displayed for invalid commands

---

2. GRAPHICAL USER INTERFACE (GUI)

Description

The Graphical User Interface (GUI) provides a user-friendly window where the user can enter commands to open Windows applications.
The GUI is developed using the Tkinter library.

The user types a command into a text field and clicks a button to execute the action.

Functionality

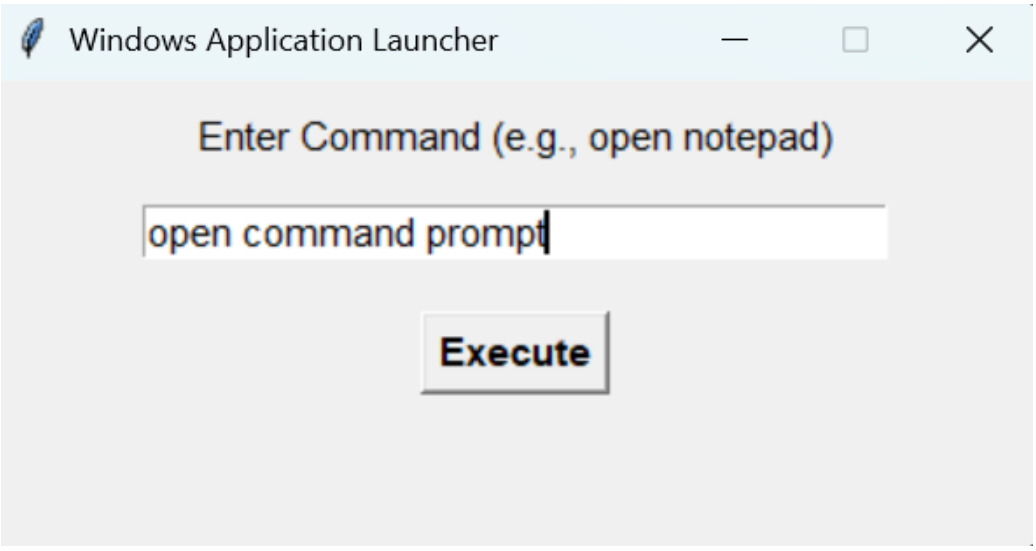
- Text box to enter command
- Execute button to process the command
- Displays error message for unsupported commands
- Opens Windows applications based on user input

PYTHON CODE:

```
import tkinter as tk

from tkinter import messagebox

import os



def execute_command():

    command = entry.get().lower().strip()



    try:

        if command == "open notepad":
```

```
            os.startfile("notepad")

        elif command == "open calculator":
            os.startfile("calc")
        elif command == "open paint":
            os.startfile("mspaint")
        elif command == "open command prompt":
            os.startfile("cmd")
        elif command == "open control panel":
            os.startfile("control")
        else:
            messagebox.showerror(
                "Invalid Command",
                "Try commands like:\nopen notepad\nopen calculator\nopen paint"
            )

    except Exception as e:
        messagebox.showerror("Error", str(e))

root = tk.Tk()
root.title("Windows Application Launcher")
root.geometry("400x180")
root.resizable(False, False)
```

```
tk.Label(
    root,
    text="Enter Command (e.g., open notepad)",
    font=("Arial", 11)
).pack(pady=10)

entry = tk.Entry(root, width=35, font=("Arial", 11))
entry.pack(pady=5)

tk.Button(
    root,
    text="Execute",
    font=("Arial", 11, "bold"),
    command=execute_command
).pack(pady=15)
root.mainloop()
```



Example Commands

- `open notepad`
- `open calculator`
- `open paint`

OUTPUT

- A window is displayed for command input
- On clicking the Execute button, the specified application opens
- Error dialog is shown for invalid commands

---

3. VOICE USER INTERFACE (VUI)

Description

The Voice User Interface (VUI) enables the user to open Windows applications using voice commands through the terminal.
Speech input is captured using a microphone and converted into text using speech recognition techniques.

The recognized voice command is displayed in the terminal and processed to perform the corresponding Windows action.

Functionality

- Listens for voice commands
- Converts speech into text
- Displays recognized command in the terminal
- Opens Windows applications based on voice input

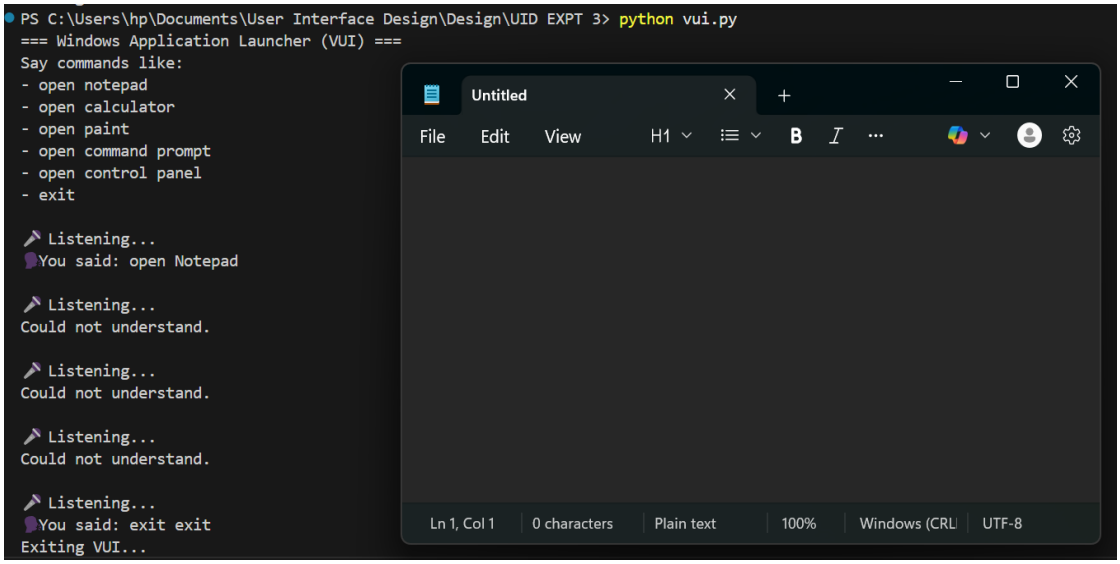
PYTHON CODE:

```
import speech_recognition as sr

import os



def open_application(command):

    command = command.lower()
```

```
if "open notepad" in command:
    os.startfile("notepad")

elif "open calculator" in command:
    os.startfile("calc")

elif "open paint" in command:
    os.startfile("mspaint")

elif "open command prompt" in command:
    os.startfile("cmd")

elif "open control panel" in command:
    os.startfile("control")

elif "exit" in command:
    print("Exiting VUI...")
    return False

else:
    print("Command not recognized.")

return True
```

```
def listen_command():
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:
        print("\n🎤 Listening...")
        recognizer.adjust_for_ambient_noise(source)
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        print("🗣 You said:", command)
        return command
    except sr.UnknownValueError:
        print("Could not understand.")
        return ""
    except sr.RequestError:
        print("Speech service error.")
        return ""



def main():
    print("=== Windows Application Launcher (VUI) ===")
    print("Say commands like:")
    print("- open notepad")
    print("- open calculator")
    print("- open paint")
```

```
    print("- open command prompt")
    print("- open control panel")
    print("- exit")

    running = True
    while running:
        command = listen_command()
        if command:
            running = open_application(command)


if __name__ == "__main__":
    main()
```



Example Voice Commands

- “Open notepad”
- “Open calculator”
- “Open paint”
- “Open command prompt”

- “Exit”

OUTPUT

- Terminal displays the recognized voice command
- Requested Windows application opens successfully
- Program exits when the user says “exit”

---

4. USER SATISFACTION COMPARISON

Description

A comparison is made between CLI, GUI, and VUI based on user satisfaction and ease of interaction.

Observation

- CLI provides fast execution for experienced users
- GUI is easy to use and visually intuitive
- VUI offers hands-free interaction but depends on voice recognition accuracy

Conclusion of Comparison

Among the three interfaces, GUI is the most user-friendly, CLI is efficient, and VUI provides an advanced interaction method.

CONCLUSION

The Windows Application Opener project successfully demonstrates how different user interfaces can be used to perform Windows actions.
The project interacts directly with the Windows operating system to launch system applications using CLI, GUI, and VUI approaches.

This experiment highlights the importance of interface design while performing operating system-level actions.